🎯 Mock Interview Report

InturnX AI Interview Assessment

Candidate Information

NAME Ankit Srivastava	EMAIL ankitsri033@gmail.com
INTERVIEWER 👩‍💼 Aditi Verma	DATE November 24, 2025
DIFFICULTY Easy	INTERVIEW TYPES DSA

68/100

Overall Performance Score

Performance Breakdown

100	95	10
HR & BEHAVIORAL	TECHNICAL	DSA

97	80	10
SYSTEM DESIGN	COMMUNICATION	CODING

Detailed Analysis

The candidate completed 8 questions in 7 minutes, demonstrating good performance overall. Communication was clear and well-structured. Technical knowledge appears strong. In the coding round, 0 out of 3 test cases passed. Continued practice in the identified areas will significantly improve interview performance.

💪 Key Strengths

- Excellent communication skills with clear and structured responses
- Provides detailed explanations and comprehensive answers
- Strong technical knowledge and understanding of concepts

📈 Areas for Improvement

- Coding solutions need improvement - practice more problems

🎯 Recommendations

- Practice more coding problems on LeetCode or HackerRank

- Focus on writing clean, efficient code with good test coverage
- Continue practicing mock interviews to build confidence
- Research the company and role thoroughly before real interviews

💻 Coding Round Results

LANGUAGE	TEST CASES PASSED
javascript	0/3

RUNTIME

65ms

📝 Interview Transcript Highlights

ADITI VERMA

Hello, I'm Aditi. I lead architecture at our company. Today, let's discuss how you think about building large-scale systems.

ADITI VERMA

Explain how you would implement rate limiting at different layers of the stack.

ADITI VERMA

How do you handle database migrations in a production system with zero downtime?

ANKIT SRIVASTAVA

I use an “expand → migrate data → contract” pattern with backward-compatible schema changes and staged deployments. That lets the old and new versions of the app run safely at the same time, so there’s no downtime. General workflow Design backward-compatible change Any schema change must work with: Current app version New app version being deployed Avoid breaking things like: Renaming/removing columns directly Making a nullable column non-nullable in one step Changing types in-place Split into multiple releases Release 1: expand schema Add new columns/tables/indexes in a way that doesn’t break current code. New fields are optional / nullable / with defaults. Background job: migrate data Copy or backfill data from old structure to new. Throttle jobs to avoid heavy locks or IO spikes. Release 2: switch application logic Deploy code that reads/writes from the new structure. Keep writing to old structure too for a while if needed (dual-write) so rollback is easy. Release 3: contract (cleanup) Once you’re confident everything is stable, drop old columns/indexes/constraints. Rolling or blue-green deploy Use a deployment strategy where instances are updated gradually: Rolling update on Kubernetes/ASG Or blue-green: bring up new version, cut traffic over with a load balancer. Because schema is backward compatible, both old & new app versions can run during the rollout. Use migration tooling Use tools like Flyway, Liquibase, Prisma migrations, Rails migrations, etc. to: Version migrations Apply in order Track which ones ran in prod For dangerous changes, mark them as manual and run during low traffic with human oversight. Minimize locking / heavy operations Avoid operations that lock huge tables in the hot path: Adding indexes concurrently/online if DB supports it. Avoid ALTER TABLE ... with type changes on big tables in a single shot. For extremely large tables: Create a new table with new schema and backfill gradually, then swap. Or use DB-specific online schema change tools. Monitoring & rollback plan Have metrics and alerts on: Error rate DB latency / deadlocks Traffic on old vs new columns/tables Rollback plan: If new code misbehaves, roll back app version. Because schema

is backward compatible and you still have old columns, rollback is safe. Quick concrete example: renaming a column Suppose we want to rename username → handle on users: Expand Add new column handle (nullable). Backfill Background job: UPDATE users SET handle = username WHERE handle IS NULL. Deploy code New app version: Reads from handle (fallback to username if needed). Writes to both username and handle (dual-write). Verify Check logs/metrics to see all reads/writes succeed, handle fully populated. Contract Stop writing username, remove fallback logic. In a later migration, drop username. At no point did we block the app or require downtime.

ADITI VERMA

What are the trade-offs of that approach?

ANKIT SRIVASTAVA

Using expand–migrate–contract keeps systems online during migrations, but it increases complexity, takes longer to deploy, requires careful monitoring, temporarily duplicates logic, and can introduce consistency risks. You trade speed for safety.

ADITI VERMA

Describe your approach to monitoring and observability in distributed systems.

ANKIT SRIVASTAVA

Monitoring and observability are critical in distributed systems because failures are often subtle and spread across many services. My approach is to design the system so we can understand what is happening internally without guessing. First, I define key metrics and SLIs like latency, throughput, error rate, and saturation for each service. I track them centrally using tools like Prometheus, Grafana, or CloudWatch. These help identify performance issues early. Second, I rely heavily on structured and centralized logging. Instead of logs scattered across servers, I collect them into a platform like ELK or Loki so I can search by trace ID across multiple services. This is important for debugging requests that hop through many components. Third, I use distributed tracing. Tools like Jaeger or OpenTelemetry let me see the full request path, measure bottlenecks, and identify where failures occur. Fourth, I set alerts and SLOs. I prefer alerts based on symptoms like user latency rather than internal technical signals. That prevents alarm fatigue and keeps focus on customer impact. Fifth, I practice observability during development, not after the fact. Every service emits consistent metadata: request IDs, correlation IDs, latency, and internal state. That means I can diagnose issues without adding logging later. Finally, I test failures with chaos engineering and load testing. Validating observability in real scenarios ensures we're not blind during outages.

ADITI VERMA

How would you design an authentication and authorization system?

ANKIT SRIVASTAVA

1. Start with goals I'd say something like: I'd design auth/authz to be secure, scalable, and easy to integrate. Concretely, I want: Strong identity verification Fine-grained access control Stateless scaling where possible Clear separation between authentication and authorization Then break it down. 2. Authentication (Who are you?) a) Identity provider & protocol Use a dedicated Auth service or an external IdP (Auth0, Cognito, Keycloak). Standard protocols: OpenID Connect / OAuth 2.0 for browser and mobile. SAML if integrating with enterprises. b) Login flows Support: Email + password SSO via Google/Microsoft etc. Optional MFA (TOTP / SMS / push). c) Credential storage & security Passwords: Hash with bcrypt/Argon2, unique salt per user. Enforce password policies and rate-limit login attempts. Sessions / tokens: Use short-lived access tokens (JWT or opaque) + refresh tokens. Store secrets/keys in a secure vault (KMS/HashiCorp Vault, etc). d) Tokens On successful login, issue: Access token: short TTL, used to call APIs. Refresh token: longer TTL, stored more securely, used to get new access tokens. If using JWT: Signed with asymmetric keys (RS256). Include user ID, roles, tenant, expiration, maybe permissions. 3. Authorization (What are you allowed to do?) a) Model I'd start with RBAC, and extend if needed: Roles (admin, student, mentor, recruiter, etc). Permissions (create_interview, view_results, manage_users). Link: users → roles → permissions. For more complex needs, consider ABAC: Rules like: "mentor can only view candidates in their own organization" using attributes (org_id, region, etc). b) Where to enforce authorization At the API gateway / edge: Basic checks: token validity, tenant, global bans, etc. Inside each microservice: Fine-grained checks based on resource ownership and business rules. c) Policy engine Centralize rules if it gets complex (e.g. OPA / Casbin). Benefits: Change policies without redeploying services. Consistent decisions across the platform. 4. System architecture Auth Service Handles login, registration, password reset, MFA. Issues and revokes tokens. Manages sessions and refresh token store (DB/Redis). API Gateway / Backend services Verify tokens on every request (signature + expiry). Extract user context (user_id, roles, tenant). Call a policy service for complex authorization decisions if needed. Token revocation / logout Use short token lifetimes. Maintain a revocation list / "token version" per user for immediate logout and compromise handling. 5. Security & best practices HTTPS everywhere. CSRF protection if using cookies (SameSite, HttpOnly, Secure). Brute-force and anomaly detection (many failed logins, new device, suspicious IP). Encrypt sensitive data at rest and in transit. Regular audits: logs for login, privilege changes, and admin actions. 6. Multi-tenancy / platforms like InturnX For a platform like InturnX: Include a tenant_id in user identity and tokens. Enforce tenant isolation on every data access. Use roles like student, mentor, company_admin, platform_admin. Use ABAC rules for "mentor can only see candidates from their org" and similar constraints.

... and more in the full transcript

InturnX

AI-Powered Mock Interview Platform

Generated on 11/24/2025, 9:11:53 PM